

Objects and classes

Victor Eijkhout, Susan Lindsey

Fall 2026

last formatted: October 8, 2026

Objects intro

Classes

1. Definition of object/class

An object is an entity that you can request to do certain things. These actions are the *methods*, and to make these possible the object probably stores data, the *members*.

When designing a class, first ask yourself:
'what functionality should the objects support'.

A class is a user-defined type; an object is an instance of that type.

2. Running example

We are going to build classes for points/lines/shapes in the plane.

```
1 class Point {  
2     /* stuff */  
3 };  
4 int main () {  
5     Point p; /* stuff */  
6 }
```

Exercise 1

Thought exercise: what are some of the actions that a point object should be capable of?

3. Object functionality

Small illustration: point objects.

Code:

```
1 // object/functionality.cpp
2 Point p(1.,2.);
3 println(
4   "distance to origin {:.6.4}",
5   p.distance_to_origin() );
6 p.scaleby(2.);
7 println(
8   "distance to origin {:.6.4}\n  and
9   angle {:.6.4}",
10  p.distance_to_origin(),
10  p.angle() );
```

Output:

```
1 distance to origin
   ↪ 2.236
2 distance to origin
   ↪ 4.472
3   and angle   1.107
```

Note the 'dot' notation.

Exercise 2

Thought exercise:

What data does the object need to store to be able to calculate angle and distance to the origin?

Is there more than one possibility?

4. The object workflow

- First define the class, with data and function members:

```
1 class MyObject {  
2     // define class members  
3     // define class methods  
4 };
```

(details later) typically before the `main`.

- You create specific objects with a declaration

```
1 MyObject  
2   object1( /* .. */ ),  
3   object2( /* .. */ );
```

- You let the objects do things:

```
1 object1.do_this();  
2 x = object2.do_that( /* ... */ );
```

5. Construct an object

The declaration of an object *x* of class *Point*; the coordinates of the point are initially set to 1.5,2.5.

```
1 class Point {  
2     private: // data members  
3         double x,y;  
4     public: // function members  
5         Point  
6             ( double x_in,double y_in ) {  
7             x = x_in; y = y_in;  
8         };  
9         /* ... */  
10 };
```

```
1 Point x(1.5, 2.5);
```

Use the constructor to create an object of a class:
function with same name as the class.
(but no return type!)

6. Private and public

Best practice we will use:

```
1 class MyClass {  
2 private:  
3     // data members  
4 public:  
5     // methods  
6 }
```

- Data is private: not visible outside of the objects.
- Methods are public: can be used in the code that uses objects.
- You can have multiple private/public sections, in any order.

Methods

7. Class methods

Definition and use of the *distance* function:

Code:

```
1 // geom/pointclass.cpp
2 class Point {
3 private:
4     float x,y;
5 public:
6     Point(float in_x,float in_y) {
7         x = in_x; y = in_y; };
8     float distance_to_origin() {
9         return sqrt( x*x + y*y );
10    };
11 };
12     /* ... */
13     Point p1(1.0,1.0);
14     float d = p1.distance_to_origin();
15     println(
16         "Distance to origin: {:.6.4}",
17         d );
```

Output:

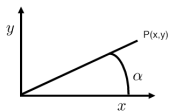
```
1 Distance to origin:
   ↪ 1.414
```

8. Class methods

- Methods look like ordinary functions,
- except that they can use the data members of the class, for instance x, y ;
- Methods can only be used on an object with the 'dot' notation. They are not independently defined.

Exercise 3

Add a method *angle* to the *Point* class. How many parameters does it need?



Hint: use the function *atan* or *atan2*.

Optional exercise 4

Make a class *GridPoint* for points that have only integer coordinates. Implement a function *manhattan_distance* which gives the distance to the origin counting how many steps horizontal plus vertical it takes to reach that point.

9. Food for thought: constructor vs data

The arguments of the constructor imply nothing about what data members are stored!

Example: create a point in where the constructor uses x, y Cartesian coordinates, but which internally stores r, θ polar coordinates:

```
1 #include <cmath>
2 class Point {
3 private: // members
4     double r, theta;
5 public: // methods
6     Point( double x, double y ) {
7         r = sqrt(x*x+y*y);
8         theta = atan2(y,x);
9     }
```

Note: no change to outward API.

Exercise 5

Discuss the pros and cons of this design:

```
1 class Point {  
2     private:  
3         double x,y,r,theta;  
4     public:  
5         Point(double xx,double yy) {  
6             x = xx; y = yy;  
7             r = // sqrt something  
8             theta = // something trig  
9         };  
10        double angle() { return theta; };  
11};
```

10. Data access in methods

You can access data members of other objects of the same type:

```
1 class Point {  
2     private:  
3         double x,y;  
4     public:  
5         void flip() {  
6             Point flipped;  
7             flipped.x = y; flipped.y = x;  
8             // more  
9         };  
10 };
```

(Normally, data members should not be accessed directly from outside an object)

Exercise 6

Extend the *Point* class of the previous exercise with a method: *distance* that computes the distance between this point and another: if p, q are *Point* objects,

$p.distance(q)$

computes the distance between them.

Hint: distance $\Delta = \sqrt{\delta_x^2 + \delta_y^2}$. Don't be afraid to introduce more methods than just *distance*.

Quiz 1

T/F?

- A class is primarily determined by the data it stores.
- A class is primarily determined by its methods.
- If you change the design of the class data, you need to change the constructor call.

11. Methods that alter the object

For instance, you may want to scale a vector by some amount:

Code:

```
1 // geom/pointscaleby.cpp
2 class Point {
3     /* ... */
4     void scaleby( float a ) {
5         x *= a; y *= a; };
6     /* ... */
7 };
8     /* ... */
9     Point p1(1.,2.);
10    println( "p1 to origin: {:.6.4}",
11        p1.distance_to_origin() );
12    p1.scaleby(2.);
13    println( "p1 to origin: {:.6.4}",
14        p1.distance_to_origin() );
```

Output:

```
1 p1 to origin:  2.236
2 p1 to origin:  4.472
```

Data initialization

12. Member default values

Class members can have default values, just like ordinary variables:

```
1 class Point {  
2 private:  
3     float x=3., y=.14;  
4 public:  
5     // et cetera  
6 }
```

Each object will have its members initialized to these values.

13. Data initialization

The naive way:

```
1 class Point {
2 private:
3     float x,y;
4 public:
5     Point( float in_x,
6           float in_y ) {
7         x = in_x; y = in_y;
8     };
```

The preferred way:

```
1 // geom/pointinit.cpp
2 class Point {
3 private:
4     float x,y;
5 public:
6     Point( float in_x,
7           float in_y )
8         : x(in_x),y(in_y) {
9     }
```

This is called a ‘member initializer list’.

There are multiple reasons for preferring this. Explanation later.
It’s technical.

Interaction between objects

14. Methods that create a new object

Code:

```
1 // geom/pointscale.cpp
2 class Point {
3     /* ... */
4     Point scale( float a ) {
5         Point scaledpoint( x*a, y*a );
6         return scaledpoint;
7     };
8     /* ... */
9     println("p1 to origin {:.5}",
10            p1.dist_to_origin());
11     Point p2 = p1.scale(2.);
12     println("p2 to origin {:.5}",
13            p2.dist_to_origin());
```

Output:

```
1 p1 to origin 2.2361
2 p2 to origin 4.4721
```

15. Anonymous objects

Create a point by scaling another point:

```
1 new_point = old_point.scale(2.81);
```

Two ways of handling the `return` statement of the `scale` method:

Naive:

```
1 // geom/pointscale.cpp
2 Point Point::scale( float a ) {
3     Point scaledpoint =
4         Point( x*a, y*a );
5     return scaledpoint;
6 };
```

Concise:

```
1 // geom/pointscale.cpp
2 Point Point::scale( float a ) {
3     return Point( x*a, y*a );
4 };
```

‘move semantics’ and ‘copy elision’:
compiler is pretty good at avoiding copies

Exercise 7

Write a method *halfway* that, given two *Point* objects *p*, *q*, construct the *Point* halfway, that is, $(p + q)/2$:

```
1 Point p(1,2.2), q(3.4,5.6);  
2 Point h = p.halfway(q);
```

You can write this function directly, or you could write functions *Add* and *Scale* and combine these.

(Later you will learn about operator overloading.)

How would you print out a *Point* to make sure you compute the halfway point correctly?

16. Using the default constructor

No constructor explicitly defined;

You recognize the default constructor in the main by the fact that an object is defined without any parameters.

Code:

```
1 // object/default.cpp
2 class IamOne {
3 private:
4     int i=1;
5 public:
6     void print() {
7         println( "{}",i );
8     };
9 };
10 /* ... */
11 IamOne one;
12 one.print();
```

Output:

1 1

17. Default constructor

Refer to *Point* definition above.

Consider this code that looks like variable declaration, but for objects:

```
1 Point p1(1.5, 2.3);  
2 Point p2;  
3 p2 = p1.scaleby(3.1);
```

Compiling gives an error (g++; different for intel):

```
1 pointdefault.cpp: In function 'int main()':  
2 pointdefault.cpp:32:21: error: no matching function for call to  
3     'Point::Point()'
```

18. Default constructor

The problem is with *p2*:

```
1 Point p1(1.5, 2.3);  
2 Point p2;
```

- *p1* is created with your explicitly given constructor;
- *p2* uses the default constructor:

```
1 Point() {};
```

- default constructor is there by default, unless you define another constructor.
- you can re-introduce the default constructor:

```
1 // geom/pointdefault.cpp  
2 Point() = default;  
3 Point( float x, float y )  
4   : x(x), y(y) {};
```

(but often you can avoid needing it)

19. Other way

State that the default constructor exists with the `default` keyword:

```
1 // object/default.cpp
2 Point() = default;
3 Point( double x, double y )
4   : x(x), y(y) {};
```

State that there should be no default constructor with the `delete` keyword:

```
Point() = delete;
```

Exercise 8

Make a class *LinearFunction* with a constructor:

```
LinearFunction( Point input_p1,Point input_p2 );
```

and a member function

```
float evaluate_at( float x );
```

which you can use as:

```
1 LinearFunction line(p1,p2);  
2 cout << "Value at 4.0: " << line.evaluate_at(4.0) << endl;
```

20. Classes for abstract objects

Objects can model fairly abstract things:

Code:

```
1 // object/stream.cpp
2 class Stream {
3 private:
4     int last_result{0};
5 public:
6     int next() {
7         return last_result++; };
8 };
9
10 int main() {
11     Stream ints;
12     println( "Next: {}",
13         ints.next() );
14     println( "Next: {}",
15         ints.next() );
16     println( "Next: {}",
17         ints.next() );
```

Output:

```
1 Next: 0
2 Next: 1
3 Next: 2
```

21. Preliminary to the following exercise

A prime number generator has:
an API of just one function: `nextprime`

To support this it needs to store:
an integer `last_prime_found`

Programming Project Exercise 9

Write a class *primegenerator* that contains:

- Methods *number_of_primes_found* and *nextprime*;
- Also write a function *isprime* that does not need to be in the class.

Your main program should look as follows:

```
1 // primes/6primesbyclass.cpp
2 cin >> nprimes;
3 primegenerator sequence;
4 while (sequence.number_of_primes_found()<nprimes) {
5     int number = sequence.nextprime();
6     cout << "Number " << number << " is prime" << '\n';
7 }
```

Programming Project Exercise 10

Write a program to test the Goldbach conjecture for the even numbers up to a bound that you read in.

First formulate the quantor structure of this statement, then translate that top-down to code, using the generator you developed above.

1. Make an outer loop over the even numbers e .
2. For each e , generate all primes p .
3. From $p + q = e$, it follows that $q = e - p$ is prime: test if that q is prime.

For each even number e then print e, p, q , for instance:

```
1 The number 10 is 3+7
```

If multiple possibilities exist, only print the first one you find.

22. A Goldbach corollary

The Goldbach conjecture says that every even number $2n$ (starting at 4), is the sum of two primes $p + q$:

$$2n = p + q.$$

Equivalently, every number n is equidistant from two primes:

$$n = \frac{p + q}{2} \quad \text{or} \quad q - n = n - p.$$

In particular this holds for each prime number:

$$\forall_{r \text{ prime}} \exists_{p, q \text{ prime}} : r = (p + q)/2 \text{ is prime.}$$

We now have the statement that each prime number is the average of two other prime numbers.

Programming Project Exercise 11

Write a program that tests this. You need at least one loop that tests all primes r , for each r you then need to find the primes p, q that are equidistant to it.

Use your prime generator. Do you use two generators for this, or is one enough? Do you need three, for p, q, r ?

For each r value, when the program finds the p, q values, print the p, q, r triple and move on to the next r .

Advanced topics

Headers

23. C headers plusplus

You know how to use `.h` files in C.

Header files for classes in C++ need some extra syntax.

24. Independent definition of function

Functions can be defined outside the class declaration:

```
1 class someclass {  
2 private:  
3     int localvar;  
4 public:  
5     // declaration:  
6     double somedo(vector);  
7 };
```

Implementation:

```
1 // definition  
2 double someclass::somedo(vector v) {  
3     .... something with v ....  
4     .... something with localvar ....  
5 };
```

25. Class declarations

Header file:

```
1 // proto/header.hpp
2 class something {
3 private:
4     int i;
5 public:
6     double dosomething( int i, char c );
7 };
```

Implementation file:

```
1 // proto/func.cpp
2 double something::dosomething( int i, char c ) {
3     // do something with i,c
4 };
```

Operator overloading

26. Operator overloading

Syntax:

```
1 <returntype> operator<op>( <argument> ) { <definition> }
```

For instance:

Code:

```
1 // geom/pointscale.cpp
2 Point operator*(float f) {
3     return Point(f*x,f*y);
4 };
5 /* ... */
6 println("p1 to origin {:.5}",
7         p1.dist_to_origin());
8 Point scale2r = p1*2.;
9 println("scaled right: {}",
10        scale2r.dist_to_origin());
11 // ILLEGAL Point scale2l = 2.*p1;
```

Output:

```
1 p1 to origin 2.2361
2 scaled right:
   ↪4.472136
```

Exercise 12

Rewrite the *halfway* method of exercise 7 and replace the *add* and *scale* functions by overloaded operators.

Hint: for the *add* function you may need '`this`'.

27. Index overloading

If you have objects that contain an array, you may want to overload the index operator.

Code:

```
1 // array/getindex4.cpp
2 class vector10 {
3 private:
4     int array[10];
5 public:
6     /* ... */
7     int operator[](int i) {
8         return array[i];
9     };
10    /* ... */
11    vector10 v;
12    v[2] = 34; // COMPILE ERROR!!!
13    cout << v[2] << '\n';
```

Output:

```
1 getindex4.cpp:38:8:
    ↪error:
    ↪expression is
    ↪not assignable
2     38 |     v[2] = 34;
    ↪// COMPILE
    ↪ERROR!!!
3         |     ~~~~ ^
4 1 error generated.
5 make[1]: ***
    ↪[../Make.inc:118:
    ↪getindex4.o]
    ↪Error 1
```

28. Index overloading'

Let the index operator return a reference:

Code:

```
1 // array/getindex5.cpp
2 class vector10 {
3 private:
4     int array[10];
5 public:
6     /* ... */
7     int& operator[](int i) {
8         return array[i];
9     };
10    /* ... */
11    vector10 v;
12    v[2] = 34;
13    cout << v[2] << '\n';
```

Output:

1 34

'This'

29. 'this' pointer to the current object

A pointer to the object itself is available as `this`. Variables of the current object can be accessed this way:

```
1 class MyClass {  
2     private:  
3         int myint;  
4     public:  
5         MyClass(int i) {  
6             this->myint = i;    // option 1  
7             (*this).myint = i; // option 2  
8         };  
9 };
```

30. 'this' use

You don't often need the `this` pointer. Example: you need to call a function inside a method that needs the object as argument)

```
1 /* forward definition: */ class someclass;
2 void somefunction(const someclass &c) {
3     /* ... */ }
4 class someclass {
5     // method:
6     void somemethod() {
7         somefunction(*this);
8     };
```

(Rare use of dereference star)

More constructors

31. Constructors and contained classes

Finally, if a class contains objects of another class,

```
1 class Inner {  
2 public:  
3     Inner(int i) { /* ... */ }  
4 };  
5 class Outer {  
6 private:  
7     Inner contained;  
8 public:  
9 };
```

32. When are contained objects created?

```
1 Outer( int n ) {  
2   contained = Inner(n);  
3 };
```

1. This first calls the default constructor
2. then calls the *Inner(n)* constructor,
3. then copies the result over the *contained* member.

```
1 Outer( int n )  
2   : contained(Inner(n)) {  
3     /* ... */  
4 };
```

1. This creates the *Inner(n)* object,
2. placed it in the *contained* member,
3. does the rest of the constructor, if any.

33. Copy constructor

- Default defined copy and 'copy assignment' constructors:

```
1 some_object x(data);  
2 some_object y = x;  
3 some_object z(x);
```

- They copy an object:
 - simple data, including pointers
 - included objects recursively.
- You can redefine them as needed.

```
1 // object/copyscalar.cpp  
2 class has_int {  
3 private:  
4     int mine{1};  
5 public:  
6     has_int(int v) {  
7         cout << "set: " << v  
8             << '\n';  
9         mine = v; };  
10    has_int( has_int &h ) {  
11        auto v = h.mine;  
12        cout << "copy: " << v  
13            << '\n';  
14        mine = v; };  
15    void printme() {  
16        cout << "I have: " << mine  
17            << '\n'; };  
18 };
```

34. Copy constructor in action

Code:

```
1 // object/copyscalar.cpp
2 has_int an_int(5);
3 has_int other_int(an_int);
4 an_int.printme();
5 other_int.printme();
6 has_int yet_other = other_int;
7 yet_other.printme();
```

Output:

```
1 set: 5
2 copy: 5
3 I have: 5
4 I have: 5
5 copy: 5
6 I have: 5
```

35. Copying is recursive

Class with a vector:

```
1 // object/copyvector.cpp
2 class has_vector {
3 private:
4     vector<int> myvector;
5 public:
6     has_vector(int v) { myvector.push_back(v); };
7     void set(int v) { myvector.at(0) = v; };
8     void printme() { cout
9         << "I have: " << myvector.at(0) << '\n'; };
10 };
```

Copying is recursive, so the copy has its own vector:

Code:

```
1 // object/copyvector.cpp
2 has_vector a_vector(5);
3 has_vector other_vector(a_vector);
4 a_vector.set(3);
5 a_vector.printme();
6 other_vector.printme();
```

Output:

```
1 I have: 3
2 I have: 5
```

36. Destructor

- Every class *myclass* has a *destructor* *~myclass* defined by default.
- The default destructor does nothing:

```
1 ~myclass() {};
```

- A destructor is called when the object goes out of scope.
Great way to prevent memory leaks: dynamic data can be released in the destructor. Also: closing files.

37. Destructor example

Just for tracing, constructor and destructor do `cout`:

```
1 // object/destructor.cpp
2 class SomeObject {
3 public:
4     SomeObject() {
5         cout << "calling the constructor"
6             << '\n';
7     };
8     ~SomeObject() {
9         cout << "calling the destructor"
10            << '\n';
11     };
12 };
```

38. Destructor example

Destructor called implicitly:

Code:

```
1 // object/destructor.cpp
2 cout << "Before the nested scope"
3     << '\n';
4 {
5     SomeObject obj;
6     cout << "Inside the nested scope"
7         << '\n';
8 }
9 cout << "After the nested scope"
10    << '\n';
```

Output:

```
1 Before the nested
   ↪scope
2 calling the
   ↪constructor
3 Inside the nested
   ↪scope
4 calling the
   ↪destructor
5 After the nested
   ↪scope
```

References and such

39. Direct alteration of internals

Return a reference to a private member:

```
1 class Point {  
2 private:  
3     double x,y;  
4 public:  
5     double &x_component() { return x; };  
6 };  
7 int main() {  
8     Point v;  
9     v.x_component() = 3.1;  
10 }
```

Only define this if you need to be able to alter the internal entity.

40. Reference to internals

Returning a reference saves you on copying.

Prevent unwanted changes by using a 'const reference'.

```
1 class Grid {
2 private:
3     vector<Point> thepoints;
4 public:
5     const vector<Point> &points() const {
6         return thepoints; };
7 };
8 int main() {
9     Grid grid;
10    cout << grid.points()[0];
11    // grid.points()[0] = whatever ILLEGAL
12 }
```

41. Access gone wrong

We make a class for points on the unit circle

```
1 // object/unit.cpp
2 class UnitCirclePoint {
3 private:
4     float x,y;
5 public:
6     UnitCirclePoint(float x) {
7         setx(x); };
8     void setx(float newx) {
9         x = newx; y = sqrt(1-x*x);
10    };
```

You don't want to be able to change just one of x,y !

In general: enforce invariants on the members.

42. Const functions

A function can be marked as const:
it does not alter class data,
only changes are through return and parameters

Static variables

43. Static class members

A static member acts as if it's shared between all objects.

(Note: C++17 syntax)

Code:

```
1 // link/static17.cpp
2 class myclass {
3 private:
4     static inline int count=0;
5 public:
6     myclass() { ++count; };
7     int create_count() {
8         return count; };
9 };
10    /* ... */
11    myclass obj1,obj2;
12    cout << "I have defined "
13         << obj1.create_count()
14         << " objects" << '\n';
```

Output:

```
1 I have defined 2
   ↪objects
```

44. Static class members, C++11 syntax

Deprecated syntax; you may come across this but do not write like this.

```
1 // link/static.cpp
2 class myclass {
3 private:
4     static int count;
5 public:
6     myclass() { ++count; };
7     int create_count() { return count; };
8 };
9     /* ... */
10 // in main program
11 int myclass::count=0;
```

Composition

45. Has-a relationship

A class usually contains data members. These can be simple types or other classes. This allows you to reflect relations between things you are modeling.

```
1 class Person {  
2     string name;  
3     ....  
4 };  
5 class Course {  
6 private:  
7     Person the_instructor;  
8     int year;  
9 };`
```

This is called the has-a relation:

Course has-a Person

46. Literal and figurative has-a

A line segment has a starting point and an end point. *LineSegment* code design:

Store both points:

```
1 class Segment {
2 private:
3     Point p_start, p_end;
4 public:
5     Point end_point() {
6         return p_end; }
7 }
8 int main() {
9     Segment seg;
10    Point somepoint =
11        seg.end_point();
```

or store one and derive the other:

```
1 class Segment {
2 private:
3     Point starting_point;
4     float length, angle;
5 public:
6     Point end_point() {
7         /* some computation
8            from the
9            starting point */ }
10 }
```

Implementation vs API: implementation can be very different from user interface.

47. Constructors in has-a case

Class for a person:

```
1 class Person {  
2 private:  
3     string name;  
4 public:  
5     Person( string name ) {  
6         /* ... */  
7     };  
8 };
```

Class for a course, which contains a person:

```
1 class Course {  
2 private:  
3     Person instructor;  
4     int enrollment;  
5 public:  
6     Course( string instr,int n ) {  
7         /* ??? */  
8     };  
9 };
```

Declare a *Course* variable as: `Course("Eijkhout",65);`

48. Constructors in the has-a case

Possible constructor:

```
1 Course( string teachername,int nstudents ) {  
2     instructor = Person(teachername);  
3     enrollment = nstudents;  
4 };
```

Preferred:

```
1 Course( string teachername,int nstudents )  
2     : instructor(Person(teachername)),  
3     enrollment(nstudents) {  
4 };
```

49. Rectangle class

To implement a rectangle with sides parallel to the x/y axes, two designs are possible. For the function:

```
1 float Rectangle::area();
```

it is most convenient to store width and height.
For inclusion testing:

```
1 bool Rectangle::contains(Point);
```

it would be convenient to store bottomleft/topright points.

For now, use the *first* option.

Exercise 13

Make a class *Rectangle* (sides parallel to axes) with a constructor:

```
Rectangle(Point botleft, float width, float height);
```

Can you figure out how to use member initializer lists for the constructors?

Implement methods:

```
float area(); float rightedge_x(); float topedge_y();
```

and write a main program to test these.

Inheritance

50. Hierarchical object relations

- Hierarchical relations between classes:
each object in class A is also in class B.

51. Example of class hierarchy

- Class *Employee*:

```
1 class Employee {  
2     private:  
3         int number, salary;  
4     /* ... */  
5 };
```

- class *Manager* is subclass of *Employee*
(every manager is an employee, with number and salary)
- Manager has extra field *n_minions*

How do we implement this?

52. Another example: multiple subclasses

- Example: both triangle and square are polygons.
- You can implement a method draw for both triangle/square
- ... or write it once for polygon, and then use that.

53. Terminology

- *Polygon* / *Employee* is the *base class*.
- *Triangle* / *Manager* is a *derived class*.
- Derived classes *inherit* data and methods from the base class: they are accessible in objects of the derived class.

54. Examples for base and derived cases

General *FunctionInterpolator* class with method *value_at*. Derived classes:

- *LagrangeInterpolator* with *add_point_and_value*;
- *HermiteInterpolator* with *add_point_and_derivative*;
- *SplineInterpolator* with *set_degree*.

55. General case, special case

You can have classes where an object of one class is a special case of the other class. You declare that as

```
1 class General {
2     protected: // note!
3     int g;
4     public:
5     void general_method() {};
6 };
7
8 class Special : public General {
9     public:
10    void special_method() { g = ... };
11 };
```

56. Inheritance: derived classes

Derived class *Special* *inherits* methods and data from base class

General:

```
1 int main() {  
2     Special special_object;  
3     special_object.general_method();  
4     special_object.special_method();  
5 }
```

Members of the base class need to be `protected`, not `private`, to be inheritable.

57. Constructors

When you run the special case constructor, usually the general constructor needs to run too. Here we invoke it explicitly:

```
1 class General {  
2 public:  
3   General( double x,double y ) {};  
4 };  
5 class Special : public General {  
6 public:  
7   Special( double x ) : General(x,x+1) {};  
8 };
```

58. Access levels

Methods and data can be

- `private`, because they are only used internally;
- `public`, because they should be usable from outside a class object, for instance in the main program;
- `protected`, because they should be usable in derived classes.

Exercise 14

Take your code where a *Rectangle* was defined from one point, width, and height.

Make a class *Square* that inherits from *Rectangle*. It should have the function *area* defined, inherited from *Rectangle*.

First ask yourself: what should the constructor of a *Square* look like?

Exercise 15

Revisit the *LinearFunction* class. Add methods *slope* and *intercept*.

Now generalize *LinearFunction* to *StraightLine* class. These two are almost the same except for vertical lines. The *slope* and *intercept* do not apply to vertical lines, so design *StraightLine* so that it stores the defining points internally. Let *LinearFunction* inherit.

59. Overriding methods

- A derived class can inherit a method from the base class.
- A derived class can define a method that the base class does not have.
- A derived class can *override* a base class method:

```
1 class Base {  
2 public:  
3     virtual f() { ... };  
4 };  
5 class Deriv : public Base {  
6 public:  
7     virtual f() override { ... };  
8 };
```

60. More

- Multiple inheritance: an X is-a A, but also is-a B.
This mechanism is somewhat dangerous.
- Virtual base class: you don't actually define a function in the base class, you only say 'any derived class has to define this function'.